

Spec — Security Architecture: Separate Public Site from Live Portal

mini-moi · Guild Created: 2026-06-14 — Claude.ai Status: spec_ready — Robert builds this week For: Claude Code + Robert (DNS/Cloudflare steps) Replaces: spec_security_audit_2026-06-14.md (superseded — root cause addressed here)

Decision: Subdomain separation

Approach is locked.

What	URL	Hosting	Always on?	Who sees it
Public site + preview	<code>minimoi.ai</code>	Cloudflare Pages	Yes	Anyone
Live portal	<code>app.minimoi.ai</code>	Cloudflare tunnel → Flask	When Mac is on	Robert + vetted guests

This is the same architecture every professional SaaS product uses (stripe.com vs dashboard.stripe.com, notion.so vs notion.so/dashboard). A solution architect reviewing this setup should recognize it as deliberate and correct — not as something that needed to be locked down after the fact.

Why not a simpler patch: Cloudflare firewall rules scoping paths on a single domain work, but they still expose the portal's existence and route structure to anyone who looks. Subdomain separation means the portal isn't reachable from the public domain at all — the attack surface for `minimoi.ai` is zero, because `minimoi.ai` is a static file host with no backend.

Defense in depth (both layers stay):

- `minimoi.ai` → static files only, no Flask process reachable
 - `app.minimoi.ai` → Flask with `@login_required` on every portal route (audit below confirms this is solid before go-live)
-

This week — task order

Follow this sequence. Each step is a prerequisite for the next.

Priority	Task	Why
1	Fix tunnel config ((<code>config.yml</code>) port 5001) + (<code>sudo cloudflared service install</code>)	Foundation — everything else depends on a stable tunnel
2	Set up Cloudflare Pages, deploy (<code>minimoi.ai</code>) as static	Big visible win, confirms always-on before touching portal
3	Add (<code>app.minimoi.ai</code>) CNAME, update tunnel config, update Flask references	Security separation
4	Remove old (<code>minimoi.ai</code>) tunnel record from Cloudflare DNS	Prevents Pages + tunnel DNS conflict
5	Run Flask security audit checklist	Hardens portal before any guest access
6	Update preview capture script output path, test refresh end-to-end	Confirms new workflow works

Gotchas (from Grok review)

Cloudflare record conflict — do not skip step 4

After step 3, the old tunnel DNS record for (`minimoi.ai`) must be explicitly removed. If both a Cloudflare Pages record and a tunnel CNAME exist for (`minimoi.ai`), they conflict — Pages may not serve correctly, or requests may intermittently route to the tunnel instead. After confirming Pages works on (`minimoi.ai`), delete any CNAME/A records still pointing (`minimoi.ai`) to the old tunnel ID.

Local development workflow

The tunnel is production-only after the split. Local development continues against (`localhost:5001`) unchanged. The gotcha is Flask config that hardcodes the domain:

- (`SERVER_NAME`): if set to (`minimoi.ai`), must change to (`app.minimoi.ai`) for production — but hardcoding breaks local dev. Solution: env var:

```
python
```

```
SERVER_NAME = os.getenv('FLASK_SERVER_NAME', None)
# Production .env: FLASK_SERVER_NAME=app.minimoi.ai
# Local: leave unset — Flask defaults to localhost:5001
```

- `SESSION_COOKIE_DOMAIN`: same pattern — set via env var for production, leave unset locally
- Login redirects: update any hardcoded domain in redirects to use `request.host` or the env var

Claude Code to grep during step 3:

```
bash
grep -r "minimoi.ai" portal/ --include="*.py" --include="*.html" --include="*.e
```

Preview capture still requires Mac

`python tools/capture_snapshot.py` hits the live Flask app to render pages — Mac must be on for the ~5 minute refresh window. Once committed and pushed, Cloudflare Pages serves the snapshot without the Mac. Expected and acceptable.

Path change — must happen in the same commit as the directory restructure:

`capture_snapshot.py` currently writes to `minimoi_portal/static/preview/`. After this migration it must write to `static/public/preview/` (repo root, where Cloudflare Pages serves from). This is a one-line path change in the script — but it must be in the **same commit** as the `static/public/` directory creation, not before or after. Writing to the old path after the restructure silently puts snapshots in the wrong place; writing to the new path before the directory exists fails. Same commit, same atomic change.

Part 1 — minimoi.ai → Cloudflare Pages (static)

What goes here

- The landing page (`index.html`) — What this is / What's running / About
- The preview section (`/preview/...`) — all captured domain snapshots
- Static assets (CSS, fonts, images) used by the landing page and preview

Nothing dynamic. No Flask. No backend calls. No login routes.

Implementation

Step 0 (investigate first): Does the landing page pull live data?

Before writing any export script, Claude Code must answer this:

```
bash

# Check the landing page route and template for dynamic data pulls
grep -r "curator\|guild\|german\|db\|query\|count\|health" \
portal/routes/main.py portal/templates/index.html 2>/dev/null
```

- **If fully static** (hand-written copy only, no DB calls, no template variables beyond static strings): just copy the template, inline the CSS, done — no export script needed.
- **If dynamic** (pulls live counts, system health, Curator stats, etc.): those data points must be stripped or hardcoded before Cloudflare Pages can serve it. Refactor the template to be fully static — the landing page copy is hand-written and should not depend on runtime data.

Answer this before starting Step 2. A dynamic landing page that silently fails on Cloudflare Pages (missing template variables → blank or broken page) is worse than pausing to investigate first.

Step 1: Export landing page as static HTML

Once confirmed static (or refactored to be so):

```
bash

# Option A: render via Flask and save output
python tools/export_landing.py # script to request / and save HTML

# Option B: if the template has no variables at all,
# just copy and inline CSS directly
```

Step 2: Set up Cloudflare Pages project

- In Cloudflare dashboard: Pages → Create a project
- Connect to the `(personal-ai-agents)` GitHub repo (public repo, branch: `(main)`)
- Build output directory: `(static/public/)` (or wherever the landing + preview static files live — Claude Code to confirm/create this path)
- Build command: none (pure static, no build step needed)
- Custom domain: `(minimoi.ai)`

Step 3: Move static files to the right directory

Create `static/public/` in the repo (or confirm the existing path):

```
static/public/  
  index.html          ← landing page  
  preview/  
    index.html        ← preview landing  
    curator/  
    german/  
    guild/  
  assets/              ← CSS, fonts, images for landing + preview  
    style.css  
    ...
```

This directory is what Cloudflare Pages serves. It's committed to the public repo — confirm no sensitive data is in any of these files before the first Cloudflare Pages deploy (landing page copy and preview content are already designed to be public).

Step 4: Update "Dashboard →" and "Request guest access →" links

In `static/public/index.html`:

- "Dashboard →" → `https://app.minimoi.ai` (or `https://app.minimoi.ai/login`)
- "Request guest access →" in preview banner → `/contact` page (also static) or Robert's LinkedIn DM link directly

Step 5: DNS update

In Cloudflare DNS:

- `minimoi.ai` A/CNAME record → Cloudflare Pages (Cloudflare handles this automatically when the custom domain is added in Pages settings)
- The existing tunnel record for `minimoi.ai` gets replaced by Pages

Part 2 — app.minimoi.ai → Cloudflare tunnel → Flask

DNS

Add a new CNAME record in Cloudflare DNS:

```
app.minimoi.ai  CNAME  <your-tunnel-id>.cfargotunnel.com
```

Or update the existing tunnel config to use `app.minimoi.ai` as the hostname instead of `minimoi.ai`.

Tunnel config update

Update `~/ .cloudflared/config.yml` (currently points to wrong port — fix this in the same pass):

```
yaml
tunnel: <tunnel-id>
credentials-file: /Users/robert/.cloudflared/<tunnel-id>.json

ingress:
  - hostname: app.minimoi.ai
    service: http://localhost:5001
  - service: http_status:404
```

Confirm `sudo cloudflared service install` runs after config update so the tunnel survives reboots.

Flask config update

Flask may have `SERVER_NAME` set or hardcoded references to `minimoi.ai` in templates or redirects. Update all references:

```
bash
# Find hardcoded domain references
grep -r "minimoi.ai" portal/ templates/ --include="*.py" --include="*.html"
```

Update any found references to use `app.minimoi.ai` where appropriate (login redirects, session cookies, CSRF config, etc.).

Flask security audit (run in same pass)

While touching Flask config, confirm the following — these are the `spec_security_audit_2026-06-14.md` findings condensed to the essentials:

- ☐ `FLASK_ENV=production` / `app.debug = False` confirmed in production config (never debug mode on the live server)
- ☐ Every portal route has `@login_required` or equivalent — run `flask routes` and spot-check any routes not obviously protected
- ☐ Admin routes (`/admin/...`, `/guild/admin`, etc.) require owner-only, not just logged-in (guest-tier users cannot reach admin)

- ☐ Custom 404 and 500 error handlers return clean pages — no stack traces, no file paths, no framework version disclosure
- ☐ Guest routes (`/guest/...`) are read-only enforced at the decorator level, not just by convention
- ☐ `/register` route: accessible but not linked from any public page (Robert shares the URL directly with vetted contacts)

Fix any findings in the same commit.

Part 3 — Preview refresh workflow (updated)

With this architecture, the monthly preview refresh is:

```
bash

# 1. Run capture script against live app (Mac must be on)
python tools/capture_snapshot.py

# 2. Output goes to static/public/preview/
# 3. Commit and push to main
git add static/public/preview/
git commit -m "content: refresh preview snapshot [Month YYYY]"
git push origin main

# 4. Cloudflare Pages auto-deploys from main — preview is live within ~60s
# No Mac needed after this point
```

The preview is now truly always-on: Cloudflare Pages serves it independently of the Mac or the tunnel.

Part 4 — Guest access (future, confirmed working)

With subdomain separation, guest access works as follows:

1. Robert decides to grant access to a specific person
2. Robert shares `https://app.minimoi.ai/register` directly (link never public)
3. Guest registers → Telegram alert to Robert → Robert approves (confirm the approval mechanism works — see guest audit in Flask security check above)
4. Guest accesses `https://app.minimoi.ai` with their credentials
5. Guest sees the read-only guest view (confirm scope)

The portal URL (`app.minimoi.ai`) is never posted publicly. It's shared person-to-person, the same way you'd share a staging environment URL.

Definition of Done

Part 1 — Cloudflare Pages:

- ☐ `static/public/` directory created with landing page + preview assets
- ☐ Landing page confirmed fully static (no dynamic data pulls)
- ☐ Cloudflare Pages project created, connected to repo, custom domain `minimoi.ai` configured
- ☐ First deploy successful — `minimoi.ai` loads the landing page
- ☐ `minimoi.ai/preview/` loads the preview index
- ☐ "Dashboard →" links to `https://app.minimoi.ai`
- ☐ No sensitive data in `static/public/` (confirm before first deploy)

Part 2 — `app.minimoi.ai`:

- ☐ `app.minimoi.ai` CNAME added in Cloudflare DNS
- ☐ Tunnel config updated to `app.minimoi.ai`, port 5001
- ☐ `sudo cloudflared service install` run — tunnel survives reboots
- ☐ Flask config updated — no hardcoded `minimoi.ai` domain references
- ☐ Flask security audit checklist complete (all items above)
- ☐ Robert confirms login works at `https://app.minimoi.ai`
- ☐ Robert confirms `minimoi.ai` (public) has no backend — works with Mac off

Part 3 — Preview refresh:

- ☐ `capture_snapshot.py` writes to `static/public/preview/`
- ☐ Push to main triggers Cloudflare Pages deploy automatically
- ☐ Confirmed: preview update doesn't require Mac to be on after push

Part 4 — Guest access:

- ☐ `/register` flow works end-to-end on `app.minimoi.ai`
- ☐ Telegram alert fires to Robert on signup
- ☐ Robert can approve guest access
- ☐ Guest session is read-only, correctly scoped

Sign-off:

- ☐ Robert reviews `minimoi.ai` as a public visitor (incognito, mobile)

- ☐ Robert reviews `app.minimoi.ai` as owner
 - ☐ Robert confirms: "safe to share preview link publicly"
-

Commit sequence

bash

```
# Part 1: static site setup
```

```
git add static/public/ .cloudflare/ cloudflare-pages.toml
```

```
git commit -m "feat: minimoi.ai → Cloudflare Pages (static public site)"
```

Landing page + preview served as static files from Cloudflare Pages.

No Flask backend on the public domain. Always-on, no Mac dependency.

'Dashboard →' updated to link to app.minimoi.ai."

```
# Part 2: portal on app subdomain + security audit
```

```
git add .cloudflared/config.yml portal/ templates/
```

```
git commit -m "feat: portal moves to app.minimoi.ai + Flask security hardening"
```

Tunnel config updated to app.minimoi.ai:5001. Flask references updated.

Security audit: debug mode confirmed off, all routes audited for auth,

admin routes owner-only, custom error pages, guest scope confirmed

read-only. Findings: [list any findings and fixes]."

Notes on professional posture

This architecture is the correct answer to "how did you secure the public preview of your personal platform?" in an interview:

"The public site is a static Cloudflare Pages deployment — there's no backend reachable from minimoi.ai at all. The live platform runs on a separate subdomain behind a Cloudflare tunnel, with Flask auth as a second layer. The preview is always on; the platform is available when I choose to make it available."

That answer demonstrates understanding of defense in depth, separation of concerns, and deliberate architecture — not post-hoc patching.
